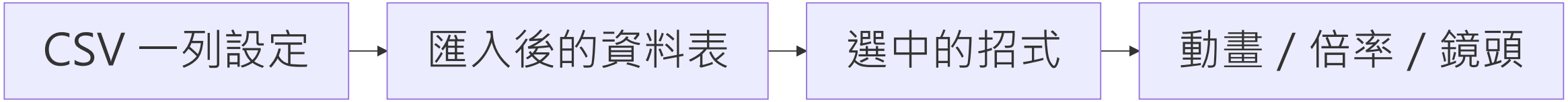
一招有一列設定

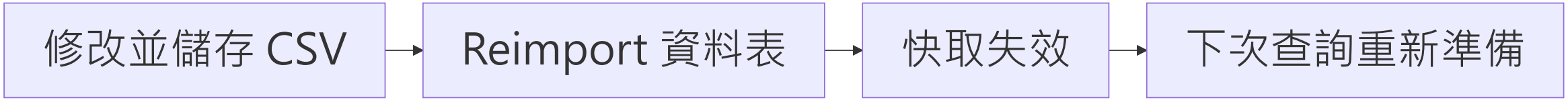


ID → Row Name；Keys → 輸入條件。
CSV 有七欄，FTndaCommandRow 宣告其餘六個資料欄。

一列招式資料，連起動畫、播放倍率與鏡頭選項。

TndaCommandData.h：FTndaCommandRow
TndaCommandReader.cpp：Prepare()

儲存設定與匯入遊戲

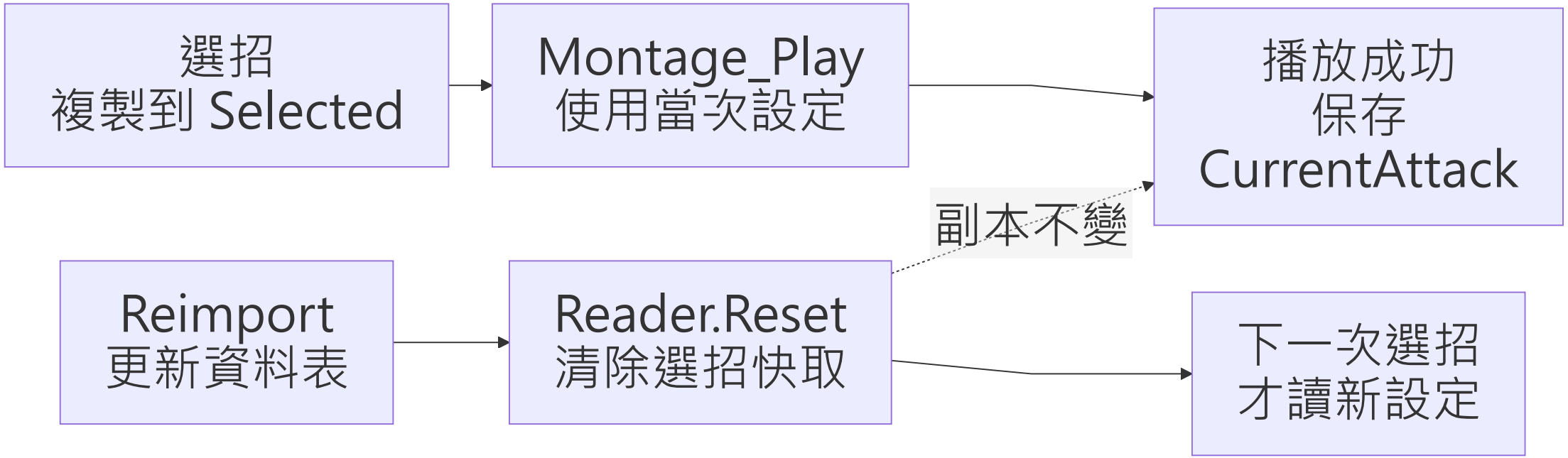


只儲存 CSV：資料表仍是舊值。
Reimport 後：快取失效，下一次查詢才重新整理。

遊戲讀的是已匯入的 DataTable；存 CSV 不等於更新遊戲資料。

TndaDataTableSubsystem.cpp：InvalidateTable()、FindPreparedTable()

匯入新設定時不會影響播放中的招式

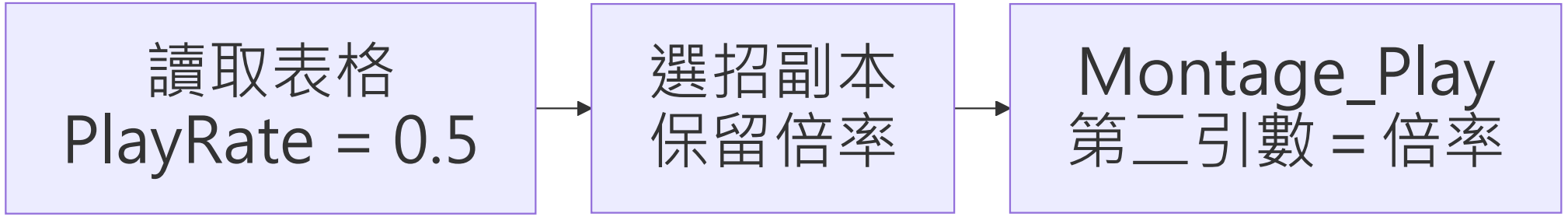


Selected 是值副本；播放成功後保存到 CurrentAttack。

有效 Reimport 不改當次副本，也不重設播放倍率；下一招才採新值。

FightingCharacter.cpp：TryExecuteCommand()、CurrentAttack
TndaCommandReader.cpp：TrySelectCommand()

CommandTable 內的 PlayRate




```
AnimInstance->Montage_Play(
    Selected.Montage, Selected.PlayRate,
    EMontagePlayReturnType::MontageLength,
    0.0f, false);
```

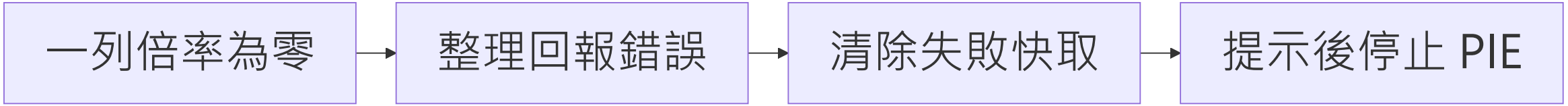
PlayRate 無單位；其他因素不變時，0.5 是半速。

理想片段時間 = 素材長度 L ÷ 倍率 r；完整解鎖另含淡出。

同一份動畫，每一列招式都能傳入自己的播放倍率。

TndaCommandReader.cpp：Prepare()、TrySelectCommand()
FightingCharacter.cpp：TryExecuteCommand() 的 Montage_Play

錯一列，整張表不可用

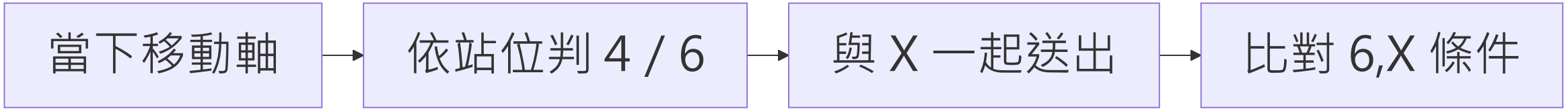


PlayRate 必須是有限且大於零的數值。
修正 CSV → Reimport → 重新啟動 PIE；完整原因看 Log 的 Row / Field。

任一列驗證失敗，整張表不可用；不釋出部分成功的招式。

TndaCommandReader.cpp：Prepare()
TndaDataTableSubsystem.cpp：ReportErrors()

方向條件來自按下攻擊的當下



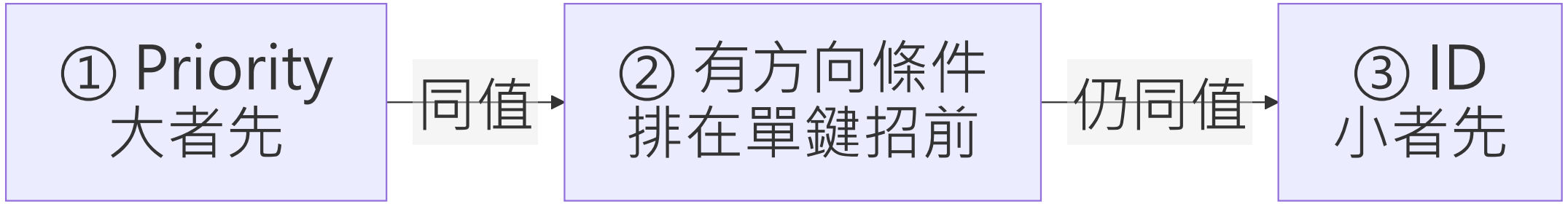
6 = 靠近對手；換邊後要重新判斷。

攻擊按下的當下讀取方向；沒有先後輸入歷史。

6,X 是方向與攻擊同時成立的條件。

FightingPlayerCharacter.cpp：Attack()、SampleAttackDirections()

選招順序與高優先度示範

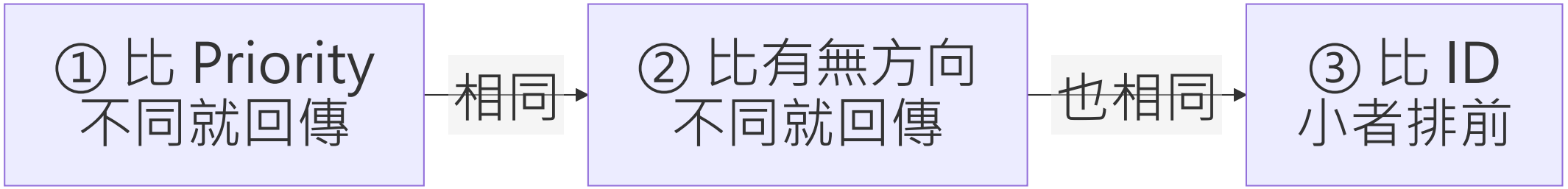


按住 6 再按 U	選中結果
兩列 Priority 都為 0	Row 2：6,X
只將 Row 1 Priority 改為 1	Row 1：X
攻擊未結束又按 U	拒絕新招，不排隊

先排序，再取第一筆符合者；Priority 不會搶佔播放中的招式。

TndaCommandReader.cpp：Prepare() 的 Commands.Sort、TrySelectCommand()
FightingCharacter.cpp：TryExecuteCommand() 入口

TArray Sort




```
Commands.Sort([](const FTndaCommand& A, const FTndaCommand& B)
{
    if (A.Priority != B.Priority)
        return A.Priority > B.Priority;
    if ((A.Direction != 0) != (B.Direction != 0))
        return A.Direction != 0;
    return A.Id < B.Id;
});
```

true 表示 A 排在 B 前；任何一層決定後就 return。

TndaCommandReader.cpp：Prepare()；TndaCommandReader.h：Commands
TArray 原地重排；[] 不擷取外部變數，const & 唯讀且不複製元素。

CameraShot｜角色提出請求




```
if (!bIsAttacking ||
    CurrentAttack.CameraShot == ECombatCameraShot::None)
    return FFightingCameraShotRequest();
return { CurrentAttack.CameraShot,
         CurrentAttack.Id, CameraShotActivationSerial };
```

請求攜帶效果選項、招式 ID 與當次啟動序號。

攻擊中且 CameraShot 不是 None，角色才提出特殊構圖請求。

TndaCommandData.h：CameraShot
FightingCharacter.cpp：GetActiveCameraShotRequest()、CurrentAttack

CameraShot｜每幀套用構圖

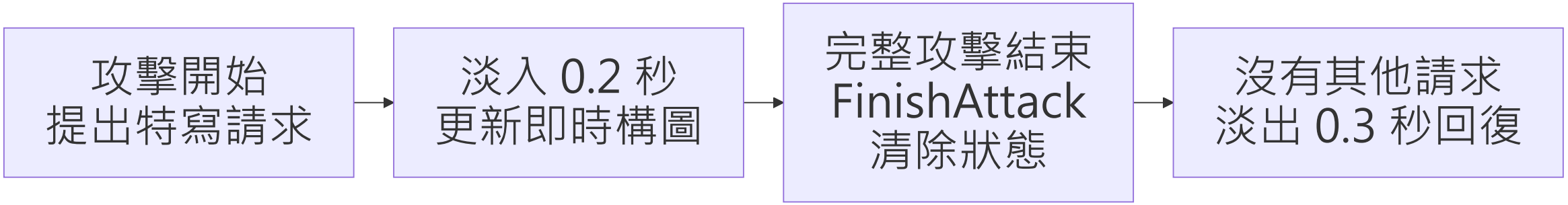


Pose 的輸出	ApplyPose 寫入的位置
FocusLocation	CameraActor 的位置
BoomRotation / TargetArmLength	SpringArm 的旋轉 / 臂長
FieldOfView	CameraComponent 的 FOV

角色提出請求，Director 計算與混合構圖，CameraActor 套用結果。

FightingCameraActor.cpp：BuildFrameInput()、Tick()、ApplyPose()
Camera/FightingCameraDirector.cpp：Update()、EvaluatePose()

CameraShot｜特寫生命週期



上半身特寫：調整雙方焦點高度與臂長，保留正常旋轉與 FOV。
效果請求涵蓋 Montage 自身淡出；回復到雙方當下的正常構圖。
雙方同時請求時，Director 比較效果優先度與啟動新舊。

取消特寫仍保留同一份動畫；Y 招不須命中就能要求特寫。

FightingCharacter.cpp：FinishAttack()、GetActiveCameraShotRequest()
Camera/FightingCameraDirector.cpp：FUpperBodyCloseUpCameraShot、BeginTransition()

CameraShot 擴展可能

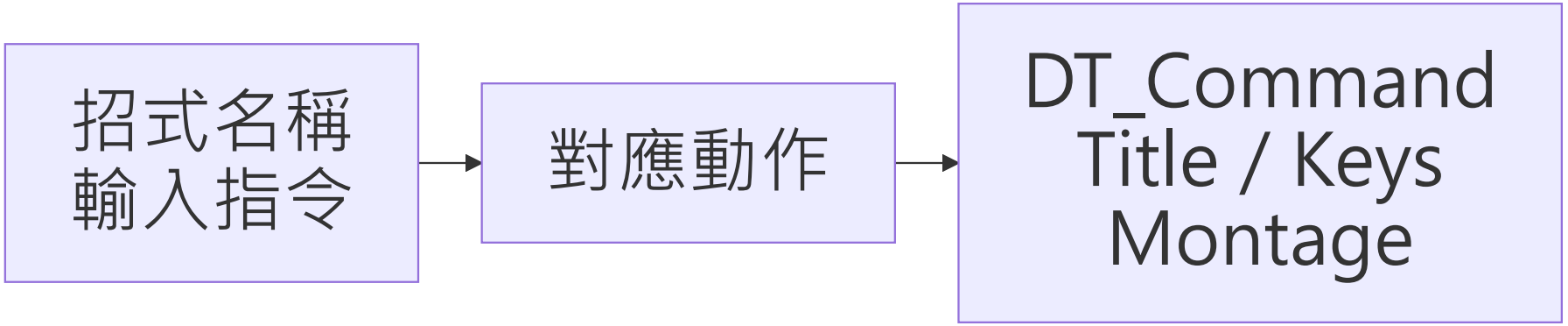
新增 enum LowAngle 示例 → 實作鏡頭介面 回傳構圖與時間 → RegisterShot 註冊選項與實作 → CSV 設定選項 Reimport 後使用

IFightingCameraShot 方法	擴充時決定什麼
Evaluate(Context)	回傳焦點 / 旋轉 / 臂長 / FOV
GetPriority()	雙方鏡頭請求的仲裁優先度
GetBlendInDuration() / GetBlendOutDuration()	效果淡入 / 淡出時間

新增選項、實作與註冊後，才可由 CSV 選用；LowAngle 尚未實作。

Camera/CombatCameraShot.h、FightingCameraShot.h、FightingCameraDirector.cpp
保留 enum 既有數值；疊加震動與命中觸發仍需另做機制。

鐵拳 8 的指令表（CommandTable）



▶ 開啟鐵拳 8 招式表影片

Tekken 8 - Jin Kazama Full Move List (Launch Version)

ToniToniVersus | 播放入口 0:00；精確暫停點待課前排練。

對照名稱、輸入條件與動作；影片不能證明外部遊戲的資料結構。

程式定位：無（外部遊戲畫面）；回教材對照 DT_Command 與 FTndaCommandRow
影片無法播放時：展示表格三列，再以 U、6+U、I 對照教材動作。